

Aluno: Matheus Bezerra Luna

Matrícula: 2023003150

Instruções:

- Esta prova possui 1 página(s). Confira se sua prova está completa.
- A clareza e concisão das respostas também é objeto de avaliação.
- As respostas podem ser a lápis.
- Podem ser utilizadas folhas de caderno como rascunho.
- Esta folha não será considerada para correção.
- Considere o grafo $G = (V, E)$ com $V(G) = \{1, 2, \dots, n\}$ e os algoritmos de percurso em largura e em profundidade vistos em sala. Considere também G representado por listas de adjacências, a menos que explícito o contrário.
- A questão bônus só será corrigida se todas as anteriores forem respondidas.

Totol: 100

Questões:

- 25 1. (25 pontos) Considere uma execução do algoritmo de **busca em profundidade** sobre um grafo conexo não direcionado $G = (V, E)$. Seja $T = (V_T, E_T)$ o grafo obtido a partir da busca, ou seja, o grafo que contém os vértices e arestas visitados durante o percurso sobre G . Mostre que T é uma árvore geradora (*árvore de profundidade*) de G .
- 15 2. (25 pontos) Seja $G = (V, E)$ um grafo conexo não direcionado e $T = (V, E_T)$ uma *árvore de profundidade* de G obtida a partir de uma busca em profundidade sobre G . Mostre que, para toda aresta $uw \in E(G)$, ou u é ancestral de w ou u é descendente de w em T .
- 25 3. (25 pontos) Seja $T = (V, E)$ uma árvore. Escreva um algoritmo que determina os valores de distâncias entre todos os pares de vértices de T (não apenas entre um vértice e os demais) em tempo $O(n^2)$? Justifique.
- 25 4. (25 pontos) Modifique o algoritmo de **busca em largura** de modo a determinar se um grafo não direcionado $G = (V, E)$ é bipartido. Note que G pode ser desconexo.
- 10 5. (Questão bônus - 20 pontos) Seja $D = (V, E)$ um digrafo (grafo direcionado). Mostre como podemos determinar as componentes fortemente conexas de D utilizando percursos em grafos. Lembre que uma componente fortemente conexa de um digrafo é um subgrafo maximal $C \subseteq D$ com respeito a propriedade de que todo vértice de C pode alcançar e ser alcançado por todo vértice de C . Não é necessário escrever um algoritmo.

